

# SEAS Miniapp Architecture

Post-refactor (7 phases). Last updated: 2026-04-11.

For detailed physics derivations and weak-form equations, see CODEBASE\_GUIDE.md. For critical numerical constraints and debug history, see CLAUDE.md.

MFEM-based Discontinuous Galerkin code for SCEC SEAS (Sequences of Earthquakes and Aseismic Slip) benchmark problems. Couples a DG elasticity domain solver with rate-and-state friction on an embedded fault interface via a quasi-dynamic approximation. Supports 2D antiplane (BP1/BP2) and 3D full elasticity (BP5).

**Code size:** ~60K LOC C++ (excluding extern/toml11).

## Module Inventory

| Module              | Directory     | Key Files                                                                                          | Purpose                                                                                 |
|---------------------|---------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Config</b>       | config/       | seas_config.hpp,<br>seas_config_parser.hpp,<br>seas_config_bridge.hpp,<br>bp{1,2,5}_params.hpp     | TOML config parsing,<br>benchmark parameters,<br>spatial functions $a(x)$ ,<br>$D_c(x)$ |
| <b>Constitutive</b> | constitutive/ | constitutive_model.hpp,<br>linear_elastic.hpp                                                      | Material model interface<br>+ isotropic linear elastic                                  |
| <b>Domain</b>       | domain/       | domain_operator.hpp,<br>elasticity_operator.hpp,<br>antiplane_operator.hpp,<br>boundary_config.hpp | DG elasticity solvers<br>(3D and antiplane),<br>boundary conditions                     |
| <b>Fault</b>        | fault/        | rate_state_fault.hpp,<br>fault_geometry.hpp,<br>fault_basis.hpp,<br>face_quadrature.hpp            | Fault ODE operator,<br>DOF management,<br>coordinate transforms                         |
| <b>Friction</b>     | friction/     | dieterich_ruina.hpp,<br>friction_law.hpp,<br>state_evolution.hpp                                   | Rate-and-state friction<br>(Brent solver), aging/slip<br>laws                           |
| <b>Integrator</b>   | integrator/   | dg_elasticity_br2_integrator.hpp,<br>dg_elasticity_ip_combined_integrator.hpp                      | DG bilinear form<br>integrators: BP2 and IP<br>methods                                  |
| <b>Solver</b>       | solver/       | seas_operator.hpp,<br>time_stepper.hpp,<br>seas_bdrload_operator.hpp                               | SEAS ODE coupling<br>operator,<br>Dormand-Prince RK45                                   |
| <b>I/O</b>          | io/           | bp5_parallel_output.hpp,<br>probe_output.hpp,<br>checkpoint.hpp,<br>paraview_output.hpp            | SCEC benchmark<br>output, checkpointing,<br>VTK visualization                           |
| <b>Common</b>       | common/       | mpi_context.hpp,<br>parallel_utils.hpp,<br>fault_scatter.hpp,<br>logging.hpp                       | MPI wrappers, parallel<br>utilities, logging                                            |
| <b>Trace</b>        | trace/        | face_trace_logger.hpp                                                                              | Face-level traction<br>diagnostics                                                      |
| <b>Driver</b>       | drivers/      | seas_driver.cpp                                                                                    | Primary BP5 parallel<br>driver<br>(TOML-configured)                                     |

| Module | Directory | Key Files                                        | Purpose                                      |
|--------|-----------|--------------------------------------------------|----------------------------------------------|
| Legacy | root      | pseas.cpp                                        | Old BP2 driver (kept for reference)          |
| Tests  | tests/    | unit/ (38), parallel/ (11),<br>verification/ (7) | Unit, MPI parallel, and full benchmark tests |

## Directory Layout

|                                               |                                                                             |
|-----------------------------------------------|-----------------------------------------------------------------------------|
| miniapps/seas/                                |                                                                             |
| ├─ drivers/                                   |                                                                             |
| │ └─ seas_driver.cpp                          | ← PRIMARY ENTRY POINT (TOML-based BP5 driver)                               |
| ├─ config/                                    | ← Configuration layer                                                       |
| │ └─ seas_config.hpp                          | SEASConfig: top-level config aggregate                                      |
| │ └─ seas_config_parser.hpp                   | TOML file parser                                                            |
| │ └─ seas_config_bridge.hpp                   | Config → domain/solver object bridge                                        |
| │ └─ bp5_params.hpp                           | BP5 parameters + spatial functions $a(x_2, x_3)$ , $D_c(x_2, x_3)$          |
| │ └─ bp5_mesh_utils.hpp                       | BP5 mesh loading utilities                                                  |
| │ └─ bp2_params.hpp                           | BP2 parameters + $a(z)$                                                     |
| │ └─ bp1_params.hpp                           | BP1 parameters                                                              |
| ├─ constitutive/                              | ← Material models                                                           |
| │ └─ constitutive_model.hpp                   | Abstract: <code>ComputeStress()</code> , <code>ComputeTangent()</code>      |
| │ └─ linear_elastic.hpp                       | $\sigma = \lambda \text{tr}(\epsilon)I + 2\mu\epsilon$                      |
| ├─ domain/                                    | ← Domain PDE solvers                                                        |
| │ └─ domain_operator.hpp                      | Abstract base: <code>Solve()</code> , <code>ComputeTraction()</code>        |
| │ └─ elasticity_operator.hpp                  | 3D DG elasticity (BR2/IP), stiffness assembly, traction                     |
| │ └─ antiplane_operator.hpp                   | 2D scalar DG (BP1/BP2)                                                      |
| │ └─ antiplane_bdrload_operator.hpp           | Boundary-load variant (BP1)                                                 |
| │ └─ boundary_config.hpp                      | BC specification (Dirichlet/Natural/Fault attrs)                            |
| │ └─ domain_config.hpp                        | Solver tuning (penalty, BLR tol, face basis)                                |
| │ └─ bp2_mesh.hpp                             | BP2 graded mesh generator                                                   |
| │ └─ seas_boundary_tags.hpp                   | Gmsh physical surface tag definitions                                       |
| ├─ integrator/                                | ← DG bilinear form integrators                                              |
| │ └─ dg_elasticity_br2_integrator.hpp         | BR2 interior + boundary (3D vector)                                         |
| │ └─ dg_elasticity_ip_combined_integrator.hpp | IP interior + boundary + traction                                           |
| │ └─ dg_elasticity_ip_penalty_integrator.hpp  | IP penalty with full elasticity tensor                                      |
| │ └─ dg_br2_integrator.hpp                    | BR2 scalar (antiplane)                                                      |
| ├─ friction/                                  | ← Friction laws                                                             |
| │ └─ friction_law.hpp                         | Abstract: <code>FrictionCoefficient()</code> , <code>SolveSlipRate()</code> |
| │ └─ dieterich_ruina.hpp                      | Regularized DR, Brent solver, psi-space                                     |
| │ └─ state_evolution.hpp                      | AgingLaw, SlipLaw, AgingLawPsi, SlipLawPsi                                  |
| ├─ fault/                                     | ← Fault interface coupling                                                  |
| │ └─ fault_geometry.hpp                       | Fault DOF management, spatial params $a(x)$ , $\eta$ , $\tau_{\text{pre}}$  |
| │ └─ rate_state_fault.hpp                     | Fault ODE operator: traction → $(V, d\psi/dt)$                              |
| │ └─ fault_basis.hpp                          | Per-face coordinate frames (normal, dip, strike)                            |
| │ └─ face_quadrature.hpp                      | Multi-DOF face basis for fault discretization                               |

|                                  |                                                   |
|----------------------------------|---------------------------------------------------|
| └─ fault_nodes.hpp               | Fault node extraction and DOF mapping             |
| └─ solver/                       | ← Top-level coupling & time integration           |
| └─ seas_operator.hpp             | SEAS quasi-dynamic ODE operator (domain + fault)  |
| └─ time_stepper.hpp              | Dormand-Prince RK45 + AdaptiveTimeStepper         |
| └─ seas_bdrload_operator.hpp     | Boundary-load coupling (BP1)                      |
| └─ io/                           | ← Output & checkpointing                          |
| └─ bp5_parallel_output.hpp       | Distributed SCEC fltst probe output               |
| └─ bp5_benchmark_output.hpp      | BP5-specific SCEC format                          |
| └─ benchmark_output.hpp          | Serial benchmark output                           |
| └─ parallel_benchmark_output.hpp | Parallel output with gather + dedup               |
| └─ probe_output.hpp              | Single-station file writer                        |
| └─ paraview_output.hpp           | VTK/ParaView diagnostic output                    |
| └─ checkpoint.hpp                | Binary checkpoint/restart                         |
| └─ common/                       | ← Shared utilities                                |
| └─ mpi_context.hpp               | MPI RAII wrapper (GlobalMax, GlobalSum, etc.)     |
| └─ parallel_utils.hpp            | MPI scatter/gather helpers                        |
| └─ fault_scatter.hpp             | Distributed fault data communication              |
| └─ logging.hpp                   | Rank-aware logging (SEAS_LOG)                     |
| └─ mpi_check.hpp                 | MPI error checking macros                         |
| └─ mpi_tags.hpp                  | MPI message tag definitions                       |
| └─ seas_types.hpp                | Enums, typedefs (BCMode, ProblemType, etc.)       |
| └─ trace/                        |                                                   |
| └─ face_trace_logger.hpp         | Per-face traction diagnostics                     |
| └─ tests/                        |                                                   |
| └─ unit/                         | (~38 tests) Fast unit tests (`make test`, ~2 min) |
| └─ parallel/                     | (~11 tests) MPI parallel tests                    |
| └─ verification/                 | (7 tests) BP1/BP2/BP5 full benchmark simulations  |
| └─ extern/toml11/                | TOML config parsing (header-only)                 |
| └─ pseas.cpp                     | Legacy BP2 driver                                 |
| └─ Makefile                      | Primary build system                              |
| └─ CMakeLists.txt                | CMake alternative                                 |
| └─ ARCHITECTURE.md               | This file                                         |
| └─ CLAUDE.md                     | Critical numerical constraints                    |
| └─ CODEBASE_GUIDE.md             | Physics-to-numerics walkthrough                   |

## Class Hierarchy

### Domain Layer

|                                                                   |                                       |
|-------------------------------------------------------------------|---------------------------------------|
| ConstitutiveModel                                                 | [constitutive/constitutive_model.hpp] |
| └─ Abstract: ComputeStress(), ComputeTangent(), GetMaxWaveSpeed() |                                       |
| └─ LinearElastic                                                  | [constitutive/linear_elastic.hpp]     |
| $\sigma = \lambda \text{tr}(\epsilon)I + 2\mu\epsilon$            |                                       |
| DomainOperator<MeshType>                                          | [domain/domain_operator.hpp]          |
| └─ Abstract: Solve(), ComputeTraction(), GetFaultDOFs()           |                                       |
| └─ MeshType = Mesh (serial)   ParMesh (parallel)                  |                                       |

- | ElasticityDomainOperator<MeshType> [domain/elasticity\_operator.hpp]
  - | 3D vector elasticity, DG BR2 or IP, ConstitutiveModel-based
- | AntiplaneDomainOperator<MeshType> [domain/antiplane\_operator.hpp]
  - | 2D scalar Laplace, DG IP or BR2

## Friction Layer

- FrictionLaw [friction/friction\_law.hpp]
  - | Abstract: FrictionCoefficient(), SolveSlipRate()
- | DieterichRuinaFriction [friction/dieterich\_ruina.hpp]
  - | Regularized DR with Brent solver
  - | Psi-space: SolveSlipRatePsi(), SolveSlipRateVectorPsi()
- StateEvolution [friction/state\_evolution.hpp]
  - | Abstract: Rate(), SteadyState()
  - | AgingLaw  $d\theta/dt = 1 - V\theta/Dc$
  - | SlipLaw  $d\theta/dt = -V\theta/Dc \cdot \ln(V\theta/Dc)$
  - | AgingLawPsi  $d\psi/dt = (bV_0/Dc) [\exp((f_0 - \psi)/b) - V/V_0]$
  - | SlipLawPsi  $d\psi/dt = -(V/Dc)(\psi - \psi_{ss})$

## Fault Layer

- FaultGeometry<MeshType> [fault/fault\_geometry.hpp]
  - | Precomputes: depths,  $a(x)$ ,  $Dc(x)$ ,  $\tau_{pre}(x)$ ,  $\eta$ ,  $V_{init}(x)$
- RateStateFaultOperator<MeshType, SlipComp> [fault/rate\_state\_fault.hpp]
  - | SlipComp = 1 (BP2 scalar) | 2 (BP5 vector)
  - | ComputeRHS(): traction  $\rightarrow$  slip rate + state evolution
- FaultBasis [fault/fault\_basis.hpp]
  - | Per-face: (normal, tangent\_dip, tangent\_strike)
  - | Embedding: (dip,strike)  $\rightarrow$  (X,Y,Z) Projection: (X,Y,Z)  $\rightarrow$  (dip,strike, $\sigma_n$ )
- FaceQuadrature [fault/face\_quadrature.hpp]
  - | Multi-DOF face basis functions for fault discretization

## Solver Layer

- TimeDependentOperator (MFEM)
  - | SEASQuasiDynamicOperator<MeshType, DomainOpType, FaultOpType>
    - | [solver/seas\_operator.hpp]
    - | Mult(): domain solve + traction + friction  $\rightarrow$  ODE RHS
    - | SetInitialCondition(): 4-phase init
    - | domain\_: DomainOpType\* (non-owning)
    - | fault\_: FaultOpType\* (non-owning)
    - | mpi\_ctx\_: MPIContext\*
    - | u\_gf\_: GridFunction (displacement solution)
    - | slip\_, traction\_, local\_slip\_, local\_traction\_
    - | elastic\_sigma\_n\_: bool (normal stress feedback, default ON)
- DormandPrinceRK45 [solver/time\_stepper.hpp]
  - | 7-stage embedded RK pair (5th/4th order), FSAL
  - | PI controller for dt adaptation
  - | V-guard: reject stages with  $V > 100 \cdot V_{stage0}$

MPI: parallel error reduction via GlobalMax

### Type Aliases (Template Instantiations)

```
// Parallel BP5 – primary production path
using PBP5SEASOp = SEASQuasiDynamicOperator<
    ParMesh,
    ElasticityDomainOperator<ParMesh>,
    RateStateFaultOperator<ParMesh, 2>>;

// Serial BP2
using SBP2SEASOp = SEASQuasiDynamicOperator<
    Mesh,
    AntiplaneDomainOperator<Mesh>,
    RateStateFaultOperator<Mesh, 1>>;
```

---

### Configuration System

Parameters flow: TOML file → CLI overrides → SEASConfig → bridge → domain/solver objects

| SEASConfig         | [config/seas_config.hpp]                                            |
|--------------------|---------------------------------------------------------------------|
| MeshConfig         | file, scale (1000.0), order (1)                                     |
| MaterialConfig     | rho (2670), cs (3464), nu (0.25) → mu(), lambda()                   |
| FrictionConfig     | V0, f0, b, L0, L_nuc, a0, amax, sigma_n                             |
| LoadingConfig      | Vp, V_init, V_nuc, delta_tau_factor, smooth_nucleation              |
| FaultGeomConfig    | Wf, lf, hs, ht, H, l_vw, w_nuc                                      |
| BoundaryTomlConfig | dirichlet_attrs, natural_attrs, fault_attr                          |
| SolverConfig       | dg_method (IP/BR2), solver_type, penalty_factor, blr_tol            |
| TimeConfig         | t_final, atol (1e-7), rtol (1e-50), max_steps, tandem_time_stepping |
| OutputConfig       | output_dir, prefix, write_every_step, ref_dir                       |
| SimulationConfig   | mode (qd / dynamic / hybrid)                                        |

Parsed by seas\_config\_parser.hpp, bridged to runtime objects by seas\_config\_bridge.hpp.

CLI overrides: --mesh, --order, --V-nuc, --delta-tau-factor, --tandem-time-stepping, etc.

---

### Execution Flow: Full Simulation (seas\_driver.cpp)

|                                                                           |                 |
|---------------------------------------------------------------------------|-----------------|
| Stage 1: CONFIGURATION                                                    | (lines 50–96)   |
| Parse TOML + CLI → SEASConfig → BP5Params, BoundaryConfig, etc.           |                 |
| Stage 2: MESH                                                             | (lines 109–154) |
| Load Gmsh file → Mesh → ParMesh (MPI partition)                           |                 |
| Stage 3: DOMAIN OPERATOR                                                  | (lines 156–172) |
| LinearElastic( $\lambda, \mu$ ) → ElasticityDomainOperator(mesh, BCs, DG) |                 |
| Assembles stiffness K once (cached, reused all steps)                     |                 |
| Stage 4: FAULT COMPONENTS                                                 | (lines 174–188) |
| FaultGeometry + DieterichRuinaFriction + AgingLawPsi                      |                 |
| → RateStateFaultOperator<ParMesh, 2>                                      |                 |

|                                                                |                 |
|----------------------------------------------------------------|-----------------|
| Stage 5: SEAS OPERATOR + INIT                                  | (lines 190–204) |
| PBP5SEASOp(domain, fault) + elastic $\sigma_n$ feedback        |                 |
| 4-phase init: PreInit → Solve → Init(equilibrium) → Verify     |                 |
| Stage 6: I/O SETUP                                             | (lines 206–241) |
| ParallelBP5BenchmarkOutput (distributed probe ownership)       |                 |
| ProbeOutput (global $V_{\max}$ log, root rank only)            |                 |
| Stage 7: TIME STEPPER                                          | (lines 252–302) |
| DormandPrinceRK45: atol=1e-7, rtol=1e-50                       |                 |
| dt_init = min(0.01·L_nuc/V_nuc, CFL), V-guard=100              |                 |
| Stage 8: TIME LOOP                                             | (lines 304–401) |
| while (t < t_final && step < max_steps):                       |                 |
| ode_solver.Step(seas_op, state, t, dt)                         |                 |
| earthquake detection: $V_{\max} > 1e-3$ m/s enter, < 1e-6 exit |                 |
| adaptive output scheduling                                     |                 |
| Stage 9: FINAL OUTPUT                                          | (lines 403–420) |
| Force write, summary: steps, rejections, earthquake count      |                 |

## Execution Flow: One ODE RHS Evaluation (Mult)

Called 7 times per RK45 step. This is the core physics cycle:

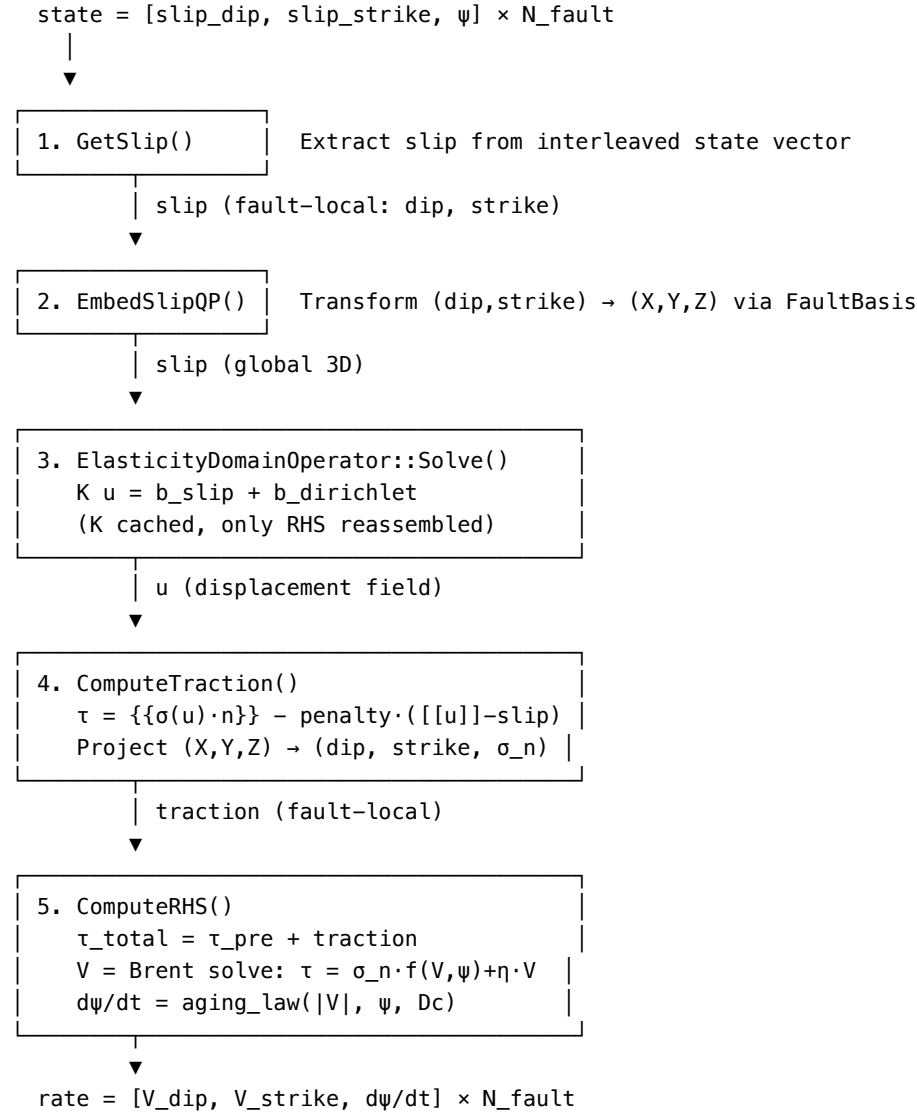
```

DormandPrinceRK45::Step(seas_op, state, t, dt)    [time_stepper.hpp]
├── For each of 7 RK stages:
│   └── seas_op.Mult(state, rate)                    [seas_operator.hpp]
│       ├── 1. fault_→GetSlip(state, slip_)
│       │   └── Extract slip from interleaved state vector
│       ├── 2. domain_→ExpandOwnedToLocalFault(slip_, local_slip_)
│       │   └── MPI: broadcast owned DOFs → ghost DOFs
│       ├── 3. domain_→Solve(t, local_slip_, u_gf_)
│       │   ├── Assemble RHS:  $b = b_{\text{slip}} + b_{\text{dirichlet}}$ 
│       │   ├── Reuse cached stiffness K
│       │   └── Solve  $K*u = b$  via MUMPS/CG+AMG
│       ├── 4. domain_→ComputeTraction(u_gf_, local_slip_, local_traction_)
│       │   ├── DG flux:  $\tau = \{\{\sigma(u) \cdot n\}\} - \text{penalty} \cdot ([u] - \text{slip})$ 
│       │   └── Project (X,Y,Z) → (dip, strike,  $\sigma_n$ )
│       ├── 5. domain_→RestrictToOwnedFault(local_traction_, traction_)
│       │   └── Select owned subset from full local traction
│       └── 6. fault_→ComputeRHS(traction_, state, rate)
│           ├── For each owned fault DOF:
│           └──  $\tau_{\text{total}} = \tau_{\text{pre}} + \text{traction}$ 

```

```
V = SolveSlipRateVectorPsi( $\tau$ ,  $\psi$ ,  $\sigma_n$ ,  $\eta$ , a) [Brent]
d $\psi$ /dt = aging_law(|V|,  $\psi$ , Dc)
```

### Data Flow Diagram



### Data Ownership

| Data               | Owner                    | Shape                      | Notes                            |
|--------------------|--------------------------|----------------------------|----------------------------------|
| Mesh (ParMesh)     | main()                   | Distributed                | MFEM standard partitioning       |
| Displacement u_gf_ | SEASQuasiDynamicOperator | Full local mesh DOFs       | GridFunction on DG FE space      |
| State vector       | main()                   | [N_owned * state_per_node] | ODE state: only owned fault DOFs |
| Slip slip_         | SEASQuasiDynamicOperator | [N_owned * slip_comps]     | Extracted from state             |

| Data                   | Owner                    | Shape                  | Notes                            |
|------------------------|--------------------------|------------------------|----------------------------------|
| Local slip local_slip_ | SEASQuasiDynamicOperator | [N_local * slip_comps] | Owned + ghosts after expand      |
| Traction traction_     | SEASQuasiDynamicOperator | [N_owned * slip_comps] | After restrict-to-owned          |
| Stiffness cached_Ah_   | ElasticityDomainOperator | Sparse matrix          | Assembled once, reused           |
| Fault geometry         | FaultGeometry            | Per-DOF arrays         | Depths, a(z), eta, Dc, tau_pre   |
| Friction constants     | DieterichRuinaFriction   | Global constants       | V0, f0, b (Dc is per-DOF in BP5) |

### State Vector Layout

**BP2** (antiplane, 1 slip component, state\_per\_node = 2):

```
state = [slip_0, theta_0, slip_1, theta_1, ..., slip_{N-1}, theta_{N-1}]
        (or psi_i if use_psi=true)
```

**BP5** (3D, 2 slip components, state\_per\_node = 3):

```
state = [slip_dip_0, slip_strike_0, psi_0, slip_dip_1, slip_strike_1, psi_1, ...]
```

Traction follows the same interleaving but without the state variable component.

## DG Formulation

### Weak Form (3D Elasticity)

Find  $u$  in  $V_h$  such that for all  $v$  in  $V_h$ :

$$\begin{aligned} & \text{sum}_K \text{integral}_K \sigma(u) : \epsilon(v) \, dx \\ & - \text{sum}_F \text{integral}_F ( \{ \sigma(u) \} \cdot [v] + \{ \sigma(v) \} \cdot [u] ) \, ds \\ & + \text{sum}_F \text{integral}_F \text{penalty} * [u] \cdot [v] \, ds \\ & = \text{integral}_{\{\Gamma_f\}} ( \dots \text{slip terms} \dots ) \, ds \\ & + \text{integral}_{\{\Gamma_D\}} ( \dots \text{Dirichlet loading} \dots ) \, ds \end{aligned}$$

where  $\sigma = \lambda \text{tr}(\epsilon)I + 2\mu\epsilon$ ,  $\{ \cdot \} = \text{average}$ ,  $[ \cdot ] = \text{jump}$ .

### SIPG Penalty (IP Method)

$$\text{penalty} = \kappa * (p+1)^2 / h * \max(\lambda + 2\mu)$$

with  $\kappa$  a user-tunable constant. The IP path applies the full elasticity tensor in the penalty term (not just a scalar).

### BR2 Method

Bassi-Rebay 2 replaces the standard penalty with a lifting operator:

$$L([u]) = M_K^{-1} * \text{integral}_e \phi * [u] \cdot n \, ds$$

The BR2 penalty term is  $\eta_{BR2} * \text{integral}_K C : L([u]) : L([v]) \, dx$ , where  $\eta_{BR2} = n_{\text{faces\_per\_element}}$  (3 for 2D triangles, 4 for 3D tets, etc.).

### Fault Interface Flux

On fault faces, the jump is modified:  $[u] - \delta$  where  $\delta$  is the prescribed slip. The traction extracted for the friction law is:

$$\tau = \{ \sigma(u) \} \cdot n - \text{penalty} * ([u] - \delta)$$



This DG-corrected traction feeds into the friction solver as the elastic loading.

---

## Friction Coupling

### Regularized Dieterich-Ruina Law

$$f(V, \psi) = a * \text{asinh}[ (V / 2V_0) * \exp(\psi / a) ]$$

where  $\psi = f_0 + b * \ln(V_0 * \theta / D_c)$  (logarithmic state variable).

### Stress Balance (Quasi-Dynamic)

$$\tau_{\text{total}} = \sigma_n * f(V, \psi) + \eta * V$$

where  $\tau_{\text{total}} = \tau_0 + \tau_{\text{elastic}}$ ,  $\eta = \mu / (2 * c_s)$  is the radiation damping coefficient.

### Slip Rate Solver

Solve  $\tau_{\text{total}} = \sigma_n * f(V, \psi) + \eta * V$  for  $V$  using Brent's method: - Bracket:  $[0, \tau/\eta]$  (monotonic, unique solution guaranteed) - BP5 vector: solve for  $|V|$  via Brent in  $\log_{10}(V)$  space, then decompose  $V_{\text{vec}} = (|V| / |\tau|) * \tau_{\text{vec}}$

### State Evolution

Aging law ( $\psi$ -space):  $d\psi/dt = (b * V_0 / D_c) * [\exp((f_0 - \psi)/b) - V/V_0]$

Steady state:  $\psi_{\text{ss}} = f_0 + b * \ln(V_0/V)$  when  $d\psi/dt = 0$ .

---

## Time Integration

Dormand-Prince RK45 (7-stage, 5th-order, FSAL) with adaptive step control:

- Error norm:  $\max_i |err_i| / (atol + rtol * |y_i|)$  (L-infinity, default)
  - PI controller:  $dt_{\text{new}} = \text{safety} * dt * err_{\text{norm}}^{-1/5}$
  - Parallel: `MPI_Allreduce(err_norm, MPI_MAX)` ensures consistent accept/reject
  - V-guard: reject RK stages where  $V_{\text{max}} > 100 * V_{\text{stage0}}$  (prevents runaway amplification)
  - Settings:  $atol = 1e-7$ ,  $rtol = 1e-50$  (pure absolute),  $dt_{\text{min}} = 1e-6$  s,  $dt_{\text{max}} = 0.1$  yr
- 

## Boundary Conditions (BP5)

Controlled by BoundaryConfig (configured via TOML or BCMODE enum):

| Mode                  | Dirichlet Faces            | Natural Faces          | Usage                 |
|-----------------------|----------------------------|------------------------|-----------------------|
| FarField<br>(default) | attrs 1-4 (X=+-Lx, Y=+-Ly) | attrs 5-6 (Z=0, Z=-Lz) | Matches Tandem        |
| XOnly                 | attrs 1-2 (X=+-Lx only)    | 3-6                    | Antiplane-style       |
| AllDirichlet          | attrs 1-6 (all)            | none                   | Legacy<br>(incorrect) |

Dirichlet loading:  $u_X = \text{sgn}(Y) * V_p * t / 2$  (along-strike plate motion).

Tandem mesh tag convention: Physical Surface 1 = Natural (top/bottom), 3 = Fault, 5 = Dirichlet (far-field).

---

## MPI Communication Pattern

Most computation is rank-local. MPI happens at these points:

| When         | Operation                 | Pattern                                               |
|--------------|---------------------------|-------------------------------------------------------|
| Per RK stage | ExpandOwnedToLocalFault() | Scatter: owned slip → ghost DOFs                      |
| Per RK stage | domain_→Solve()           | MUMPS distributed solve (internal MFEM/Hypre)         |
| Per RK stage | fault_→ComputeRHS()       | Purely local (no MPI)                                 |
| Per RK step  | Error norm reduction      | MPI_Allreduce(MAX) — consistent accept/reject         |
| Per RK step  | Max slip rate             | MPI_Allreduce(MAX) — earthquake detection, dt control |
| At I/O setup | Probe station ownership   | MPI_Allreduce(MIN) on distances (computed once)       |
| At output    | Station writes            | Each rank writes only its owned stations              |

---

**Critical:** The error norm MPI\_Allreduce(MAX) prevents rank divergence; without it, ranks disagree on accept/reject and deadlock.

---

## Coordinate System (BP5)

SCEC Convention: Tandem/MFEM Mesh:  
 x1 = fault-normal → Y (fault at Y=0)  
 x2 = along-strike → X  
 x3 = depth (+down) → -Z (Z=0 surface, Z<0 depth)

Fault-local frame (FaultBasis): - tangent1 = dip direction = (0, 0, +1) (downward into earth) - tangent2 = strike direction = (1, 0, 0) (along-strike, dominant slip) - normal = (0, 1, 0) or (0, -1, 0) depending on face orientation

---

## Initialization Sequence (4-Phase)

1. **PreInit:** slip=0,  $\psi = f_0 + b \cdot \ln(V_0/V_{\text{init}})$  (steady-state placeholder)
  2. **First domain solve:**  $K \cdot u = b$  with zero slip, compute initial elastic traction
  3. **Init:** Compute  $\psi(0)$  from stress equilibrium:  $\tau_{\text{slip}} + \tau_{\text{elastic}} = \sigma_n \cdot f(V_{\text{init}}, \psi) + \eta \cdot V_{\text{init}}$
  4. **Verification re-solve:** Re-solve domain, re-compute traction, verify equilibrium error < 1e-6
- 

## BP5 Spatial Parameters

| Parameter | VW Core | VS Zone | Transition   | Nucleation Zone |
|-----------|---------|---------|--------------|-----------------|
| a         | 0.004   | 0.04    | linear blend | 0.004           |
| b         | 0.03    | 0.03    | 0.03         | 0.03            |
| Dc        | 0.14 m  | 0.14 m  | 0.14 m       | 0.13 m          |

| Parameter | VW Core | VS Zone | Transition | Nucleation Zone      |
|-----------|---------|---------|------------|----------------------|
| sigma_n   | 25 MPa  | 25 MPa  | 25 MPa     | 25 MPa               |
| V_init    | V_init  | V_init  | V_init     | V_nuc (configurable) |

VW core:  $hs+ht \leq \text{depth} \leq hs+ht+H$  AND  $|x2| \leq l_{vw}/2$  (depth 4-16 km, +/-30 km strike).

Nucleation zone: left edge of VW,  $-l_{vw}/2 \leq x2 \leq -l_{vw}/2 + w_{nuc}$ , same depth range.

## Key Physics Equations Summary

| Equation                   | Formula                                                                        | Code Location                                |
|----------------------------|--------------------------------------------------------------------------------|----------------------------------------------|
| Momentum balance           | $\square \cdot \sigma(u) = 0$                                                  | elasticity_operator.hpp Solve()              |
| Constitutive law           | $\sigma = \lambda \text{tr}(\epsilon)I + 2\mu\epsilon$                         | linear_elastic.hpp ComputeStress()           |
| Fault stress balance       | $\tau = \sigma_n \cdot f(V, \psi) + \eta \cdot V$                              | dieterich_ruina.hpp SolveSlipRatePsi()       |
| Friction coefficient       | $f =$<br>$a \cdot \text{asinh}[(V/2V\square) \cdot \exp(\psi/a)]$              | dieterich_ruina.hpp FrictionCoefficientPsi() |
| Aging law ( $\psi$ -space) | $d\psi/dt =$<br>$(bV\square/Dc)[\exp((f\square - \psi)/b) - V/V\square]$       | state_evolution.hpp AgingLawPsi::Rate()      |
| Radiation damping          | $\eta = \mu/(2c_s)$                                                            | fault_geometry.hpp constructor               |
| DG traction                | $\tau = \{ \{ \sigma \cdot n \} \} - \text{penalty} \cdot ([u]) - \text{slip}$ | elasticity_operator.hpp ComputeTraction()    |

## Building and Testing

```
conda activate mfem-dev                                # Build environment

# Build
make all                                                # Everything
make seas_driver                                       # Primary BP5 driver

# Unit tests (fast, ~2 min)
make test

# Parallel tests
mpirun -np 4 seas_test_bp5_parallel_smoke
mpirun -np 8 seas_test_parallel_elasticity

# Full verification (long-running)
mpirun -np 8 seas_bp5_full --config bp5/config/bp5_1000m.toml
```

## Known Limitations

1. **Matrix assembly:** Stiffness matrix assembled and stored (not matrix-free). Memory-bound for large meshes at high polynomial order.
2. **Direct solver default:** MUMPS (or MUMPS\_BLR) for BP5. Iterative solvers (CG+AMG) available but less robust for DG.

3. **Single fault plane:** Code assumes one planar fault at  $Y=0$ . Multi-fault or non-planar faults not supported.
4. **Quasi-dynamic only:** No fully dynamic wave propagation. Radiation damping ( $\eta \cdot V$ ) approximates dynamic effects. (Note: feature/elasticity-inertia branch in progress.)
5. **DG noise on unstructured meshes:** BR2 traction can show  $O(h)$  noise at sharp element-size transitions. Smooth mesh grading recommended.